

Cheatsheet — conversioni per il mapping (CALC)

Da tenere aperto mentre si studia e da rifare a memoria prima della prova: all'esame non si porta niente (dispensa 01 p. 8). Ogni regola ha un esempio già verificato. Nasce dalla sessione del 2026-10-08 (lezione [[lezione_02p_fondamenta]]); le regole di conto ridotte all'osso sono anche in [[prontuario_CALC]] §0.

1. Potenze di due da sapere a memoria

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}
1	2	4	8	16	32	64	128	256	512	1024

Le unità: ogni gradino moltiplica per 1024, cioè **aggiunge 10 all'esponente**.

K (kilo)	M (mega)	G (giga)
2^{10}	2^{20}	2^{30}

2. Da una taglia ai bit interni (K)

Regola: spezza in *numero* × *unità*, scrivi entrambi come potenza di due, **somma gli esponenti**.

$$\begin{aligned} 32 \text{ KB} &= 2^5 \times 2^{10} = 2^{15} && \rightarrow K = 15 \\ 512 \text{ KB} &= 2^9 \times 2^{10} = 2^{19} && \rightarrow K = 19 \\ 64 \text{ MB} &= 2^6 \times 2^{20} = 2^{26} && \rightarrow K = 26 \\ 2 \text{ GB} &= 2^1 \times 2^{30} = 2^{31} && \rightarrow K = 31 \end{aligned}$$

- **K = numero di piedini d'indirizzo del chip** ($A[K-1..0]$).
- **32 – K = bit che scelgono il blocco** (nel DLX, che ha 32 bit). 2 GB → resta 1 bit: BA31.

⚠ Non dimenticare l'unità: «32 KB → 5» è sbagliato, si scrivono **sempre due** potenze.

3. Da binario a decimale

Regola: ogni posizione vale il doppio di quella alla sua destra. Somma i pesi dove c'è 1.

$$\begin{array}{rcll} \text{pesi:} & 8 & 4 & 2 & 1 \\ & 1 & 1 & 0 & 0 & \rightarrow & 8 + 4 & = & 12 \\ & 1 & 1 & 0 & 1 & \rightarrow & 8 + 4 + 1 & = & 13 \end{array}$$

$$\begin{array}{ccccccccc} 16 & 8 & & 4 & & 2 & & 1 & \\ 1 & 0 & & 1 & & 1 & & 0 & \end{array} \rightarrow 16 + 4 + 2 = 22$$

Controllo: rifai la somma partendo da destra. 1100 non è 8: c'è anche il 4.

4. Da decimale a binario

Regola: togli la potenza di due più grande che ci sta, metti 1 in quella posizione, ripeti col resto. Dove non togli niente, metti 0.

12: ci sta 8 → resto 4; ci sta 4 → resto 0 → 8 4 2 1 = 1 1 0 0
 6: non ci sta 8; ci sta 4 → resto 2; ci sta 2 → 0 → 8 4 2 1 = 0 1 1 0

5. Esadecimale ↔ binario

Una cifra esadecimale = 4 bit esatti. La h in fondo vuol dire «esadecimale».

0 0000	4 0100	8 1000	C 1100
1 0001	5 0101	9 1001	D 1101
2 0010	6 0110	A 1010	E 1110
3 0011	7 0111	B 1011	F 1111

Trucco per ricostruire la tabella: A = 10, B = 11, ... F = 15, e poi la conversione del §4.

Da hex a binario: sostituisci ogni cifra con i suoi 4 bit.

2800h = 2 8 0 0 = 0010 1000 0000 0000
 C000h = C 0 0 0 = 1100 0000 0000 0000

Da binario a hex: dividi in gruppi di 4 **partendo da destra**, poi converti ogni gruppo.

0010 0111 1111 1111 = 2 7 F F = 27FFh

Indirizzi del DLX: 32 bit = **8 cifre** hex. Il bit più a sinistra, BA31, è il primo bit della prima cifra.

80000000h = 1000 0000 ... 0000 → solo BA31 = 1
 40000000h = 0100 0000 ... 0000 → solo BA30 = 1

6. Le taglie in esadecimale (DLX)

La taglia scritta in hex è **anche l'indirizzo dove finisce il blocco che parte da 0, più uno.**

Taglia	Potenza	In hex
1 MB	2^{20}	00100000h
16 MB	2^{24}	01000000h

Taglia	Potenza	In hex
64 MB	2^{26}	04000000h
128 MB	2^{27}	08000000h
256 MB	2^{28}	10000000h
512 MB	2^{29}	20000000h
1 GB	2^{30}	40000000h
2 GB	2^{31}	80000000h

Come ricavarle senza tabella: dividi l'esponente per 4. Il **quoziente** è il numero di zeri in fondo, il **resto** dice la cifra davanti (resto 0 → 1, 1 → 2, 2 → 4, 3 → 8).

2^{30} : $30 = 4 \times 7 + 2 \rightarrow$ cifra 4, poi 7 zeri → 40000000h

2^{26} : $26 = 4 \times 6 + 2 \rightarrow$ cifra 4, poi 6 zeri → 04000000h (8 cifre: si riempie a sinistra con

7. Fine di un blocco

Regola: fine = inizio + taglia – 1. In binario: gli **ultimi K bit tutti a 1**.

1 GB da 00000000h: $00000000h + 40000000h - 1 = 3FFFFFFh$

512 MB da 40000000h: $40000000h + 20000000h - 1 = 5FFFFFFh$

64 MB da 60000000h: $60000000h + 04000000h - 1 = 63FFFFFFh$

1 GB da 80000000h: $80000000h + 40000000h - 1 = BFFFFFFh$

Il «– 1» c'è perché si conta **da 0**: 4 celle sono 0, 1, 2, 3, e l'ultima è la 3.

8. Allineamento: il blocco può partire da qui?

Regola: un blocco da 2^K parte da un indirizzo allineato se **gli ultimi K bit sono 0**, cioè se l'indirizzo è un multiplo della taglia.

Scorciatoie (dispensa 02 p. 38): - da 2 byte: indirizzo **pari** (ultimo bit 0); - da 8 byte: ultimi **3** bit a 0; - da 16 byte: ultima **cifra hex** a 0; - in generale, se K è multiplo di 4: le ultime **K/4 cifre hex** a 0.

64 MB (K = 26) a 60000000h? $60000000h = 0110\ 0000 \dots$: dopo il bit 29 è tutto 0 → sì
 512 MB (K = 29) a 04000000h? $04000000h = 0000\ 0100 \dots$: il bit 26 è 1, sotto i 29 → no

Per non sbagliare: disponi i blocchi **dal più grande al più piccolo** partendo da 0, e l'allineamento viene da solo (lezione 02, Decisione 1).

9. Bit alti e bit bassi

«Alti» = le cifre **a sinistra** nella scritta del numero, che pesano di più. «Bassi» = quelle **a destra**.

- i **K bit bassi** scelgono la **cella** dentro il chip (vanno ai piedini A[K-1..0]);
- i bit **alti** che restano scelgono **quale chip**: sono quelli che usi nel CS.

esempio con 3 bit e chip da 4 celle (K = 2):

1		0	1
chip		cella	

Prova veloce (soluzioni sotto)

1. Bit interni di una EPROM da 256 KB.
2. C000h in binario. Quali bit sono a 1?
3. Fine di un blocco da 256 MB che parte da 00000000h.
4. Un blocco da 1 GB può partire da 60000000h?

Soluzioni

1. $256 \text{ KB} = 2^8 \times 2^{10} = 2^{18} \rightarrow \mathbf{18}$.
2. 1100 0000 0000 0000: sono a 1 i due bit più a sinistra (A15 e A14).
3. $256 \text{ MB} = 100000000h \rightarrow \text{fine} = 0FFFFFFFh$.
4. $1 \text{ GB} = 2^{30}$: servono gli ultimi 30 bit a 0. $60000000h = 0110 \text{ } 0000 \dots$ ha il bit 29 a 1 $\rightarrow \mathbf{no}$. Partenze valide: 00000000h, 40000000h, 80000000h, C0000000h.